

ChargeWise AI

Relatório de evolução do assistente para gestão de recarga de veículos elétricos em condomínios

Identificação

Curso: Ciência da Computação
Turma: 2º semestre (2SEM) | 2026.2
Disciplina: Prompt and Artificial Intelligence
Professor: Jorge Luiz Gomes
Parceiro: GoodWe Brasil

Propósito

Apoiar moradores e gestores em consultas sobre carregadores GoodWe, status operacional, potência, faturamento e sustentabilidade, com respostas seguras e consistentes.

Integrantes

Bernardo Zauza Amorim	RM 568808
Bruno Almeida de Oliveira	RM 572648
Gabriel Góes Nunes Pereira	RM 571735
Guilherme Vinciguerra Carvalho	RM 571951
Marcos Peterson Martins Pereira	RM 573857
Matheus Jorge Santana	RM 574166

Resumo das Sprints 1/2

A etapa inicial definiu escopo e identidade do assistente. A Sprint 2 entregou um chatbot funcional com prompt textual, contexto condominial e histórico mantido em uma lista Python. Seis cenários documentados foram considerados adequados, mas a solução ainda produzia texto livre, reenviava todo o histórico e dependia do próprio modelo para aplicar regras de segurança.

Objetivo da Sprint 03

Evoluir a prova de conceito para uma arquitetura LCEL modular, com memória por sessão e limite de tokens, saída estruturada validada por Pydantic, guardrails antes e depois do modelo, prompts versionados e avaliação reproduzível de qualidade, custo e latência.

Refatoração para LCEL

O fluxo monolítico das Sprints 1/2 foi substituído por componentes com responsabilidades explícitas. O *GoodWeChatbot* delega à cadeia LCEL; cada sessão recebe histórico próprio; o modelo é chamado apenas para solicitações permitidas; e toda saída passa por parsing e moderação.



Solicitações fora de escopo, jailbreak, fraude ou risco elétrico são encerradas pelo guardrail local, sem consumir tokens do LLM.

Decisões técnicas

- **LCEL:** composição declarativa com *ChatPromptTemplate*, *MessagesPlaceholder*, *RunnableLambda* e *RunnableWithMessageHistory*.
- **Schema Pydantic v2:** contrato único para intenção, resposta, estado, potência, valor, encaminhamento e confiança.
- **Memória por sessão:** limite de 4.096 tokens e resumo extrativo determinístico dos turnos antigos.
- **Defesa em camadas:** validação determinística antes do modelo e moderação após a geração.
- **Prompt versionado:** v4 atual com tags XML, saída JSON e contexto condominial delimitado.

Trade-offs assumidos

- Regex é rápida, previsível e barata, porém pode gerar falsos positivos ou não cobrir novas formulações.
- Schema rígido aumenta confiabilidade de integração, mas uma resposta útil com JSON inválido precisa cair em fallback.
- Resumo sem LLM não adiciona custo nem latência de modelo, mas pode eliminar nuances importantes.
- Memória em processo simplifica a PoC, porém não persiste após reinício e não atende escalabilidade horizontal.
- O 120B apresentou maior qualidade no dataset avaliado; o 20B continua útil quando o custo tiver prioridade.

Resultado arquitetural: o dataset continha 15 casos de recusa esperada. Na execução inicial, um follow-up contextual adicional foi bloqueado indevidamente, totalizando 16 bloqueios locais e 19 chamadas ao modelo, contra 35 no legado. A correção posterior passou a permitir esse tipo de referência contextual.

Comparação antes e depois

A comparação canônica reexecutou legado e LCEL no mesmo dataset de 35 casos, com o modelo real **gpt-oss:120b**. O legado utilizou o prompt v1 e a Sprint 03, o prompt v3.

Métrica	Sprints 1/2	Sprint 03 LCEL
Qualidade média	7,09 / 10	8,97 / 10
Tokens por turno/caso*	1.720,80	859,40
Tokens por chamada ao modelo	1.720,80	1.583,11
Latência média ponta a ponta	3.096,85 ms	892,33 ms
Structured output válido	0,00%	94,29%
Recusas corretas	66,67%	100,00%
Memória contextual	66,67% real	33,33% real; 100% corrigido offline

* Tokens por turno/caso = tokens totais divididos pelos 35 casos, incluindo recusas locais com zero token de modelo. A linha seguinte usa somente as chamadas efetivas. "Sprints 1/2" representa o legado reavaliado, não os seis testes manuais históricos.

Evolução da memória contextual

Sprint 01	Sprint 02	Sprint 03 inicial	Sprint 03 corrigida
Não implementada	66,67% automatizado*	33,33%	100,00%

* A Sprint 02 aprovou seu único cenário manual de memória. No protocolo automatizado posterior, recuperou semanticamente 3/3 referências, mas uma delas foi registrada como falha porque "GW-123" usou hífen Unicode; por isso o artefato histórico mostra 66,67%. A Sprint 03 corrigida acertou 3/3 follow-ups na avaliação técnica offline: todos chegaram ao modelo e receberam o histórico. A comparação com modelos reais permanece como medição histórica até nova execução autenticada.

+1,88

pontos de qualidade

-50,1%

tokens totais
60.228 → 30.079

35 → 19

chamadas ao modelo

100%

encaminhamento profissional

Ganhos comprovados

- Saída estruturada passou de inexistente para 94,29% de validade.
- Recusas corretas cresceram 33,33 pontos percentuais.
- Encaminhamento profissional subiu de 40% para 100%.
- Os 16 bloqueios locais reduziram em 45,7% as chamadas ao modelo.

Regressões observadas

- A regressão inicial de memória foi corrigida: 33,33% → 100,00% no teste offline.
- Happy paths caíram de 86,67% para 80%.
- Nas chamadas reais, a latência do modelo aumentou de 801,48 para 1.388,56 ms.

Leitura correta: a queda da latência ponta a ponta é explicada sobretudo pelas recusas locais quase imediatas. A memória não era perdida pelo buffer: follow-ups pronominais eram bloqueados antes do modelo. Após tornar o guardrail sensível ao contexto e corrigir a normalização da métrica, os três casos avaliáveis passaram.

Problemas, alternativas e decisões

1. Segurança dependente do comportamento probabilístico

Problema: no legado, pedidos fora de escopo ou perigosos sempre chegavam ao modelo e dependiam da obediência ao prompt.

Solução escolhida: guardrail determinístico antes do LLM, com recusa específica para escopo, jailbreak, fraude, aconselhamento impróprio e risco elétrico; moderação posterior permanece como segunda barreira.

Alternativas: somente reforçar o system prompt; usar outro LLM como classificador; depender exclusivamente de moderação posterior.

Motivo: bloqueio local é auditável, tem baixa latência e custo zero de modelo. O resultado foi 100% de recusas corretas e 16 chamadas evitadas em relação ao legado.

2. Histórico ilimitado e sem isolamento entre sessões

Problema: a lista Python reenviava todo o histórico; o crescimento era ilimitado e uma instância não separava conversas por `session_id`.

Solução escolhida: `RunnableWithMessageHistory` e memória por sessão com orçamento de tokens, retenção de turnos recentes e resumo extrativo dos antigos.

Alternativas: truncamento simples; resumo por LLM; banco persistente; LangGraph, experimentado durante a migração.

Motivo: a investigação mostrou que o histórico permanecia disponível, mas o guardrail bloqueava “dele” e “ela” antes da chamada ao modelo. A validação passou a aceitar somente follow-ups curtos com referência explícita e contexto recente do domínio, preservando o bloqueio de solicitações não relacionadas. O resultado offline evoluiu de 33,33% para 100,00%.

3. Texto livre sem contrato de integração

Problema: respostas livres não garantiam campos, tipos ou coerência semântica para consumo por outros sistemas.

Solução escolhida: JSON compatível com *ConsultaRecarga*, validação Pydantic v2, proibição de campos extras e fallback seguro.

Alternativas: parsing por regex; JSON sem schema; aceitar correção silenciosa de qualquer saída.

Motivo: contrato explícito falha de forma segura e elevou a validade estrutural de 0% para 94,29%, embora respostas úteis malformadas possam ser descartadas.

4. Modelo comparativo originalmente indisponível

Problema: *qwen3:8b* não aparecia no endpoint autenticado em 12/09/2026.

Solução escolhida: usar *gpt-oss:20b*, disponível no mesmo endpoint, mantendo dataset e parâmetros iguais.

Alternativas: trocar de provedor; usar um mock; aguardar disponibilidade.

Motivo: priorizar uma comparação real e reproduzível, registrando a limitação em vez de simular resultados.

Modelos, prompts e trabalho do grupo

Comparação de modelos

Métrica	120B	20B
Qualidade	9,52	8,37
Intenção	91,43%	77,14%
Structured output	98,57%	87,15%
Latência do modelo	1.247,73 ms	3.381,32 ms
Custo/repetição	US\$ 0,00706	US\$ 0,00370

Foram feitas duas repetições de 35 casos por modelo. Nesse dataset e nessas execuções, o **gpt-oss:120b** apresentou maior qualidade, estabilidade e menor latência. O 20B, cerca de 47,5% mais barato, fica como contingência quando custo superar confiabilidade. A amostra é adequada ao exercício, mas não constitui SLA.

Evolução dos prompts

- v1:** baseline textual extenso, regras e exemplos.
- v2:** tags XML, escopo GoodWe/EV e contrato JSON.
- v3:** defesa ampliada contra jailbreak, fraude e ataques indiretos; recusas oficiais e indicação de profissional.
- v4:** contexto condominial delimitado, precedência de dados da simulação e separação entre dados operacionais e especificações oficiais. Sua avaliação suplementar offline não é diretamente comparável ao baseline canônico.

No ensaio offline determinístico, v3 usou 924,43 tokens/caso, 49,99% menos que v1. Como todas as versões atingiram 100% nesse mock, o ensaio comprova redução de tokens, não ganho real de qualidade.

Fontes internas: *evals/before_after.md*, *evals/legacy_results.json*, *evals/sprint3_results.json*, *docs/relatorio_modelos.md*, *prompts/README.md* e código em *src/*. Métricas coletadas em 11–12/09/2026.

Equipe

Os nomes completos e RMs dos seis integrantes estão apresentados na identificação da página 1.

Conclusão

A Sprint 03 transformou um chatbot de texto livre em um fluxo modular, mensurável e defensivo. Na memória, a evolução foi de ausência na Sprint 01 e histórico manual na Sprint 02 para sessões isoladas e limitadas por tokens na Sprint 03. A correção do roteamento contextual elevou o teste offline de 33,33% para 100,00% sem reduzir as recusas corretas.

Próximos passos da memória: ampliar os três follow-ups avaliáveis; validar conversas próximas do limite de tokens; persistir sessões fora do processo; e repetir a medição com os modelos reais.